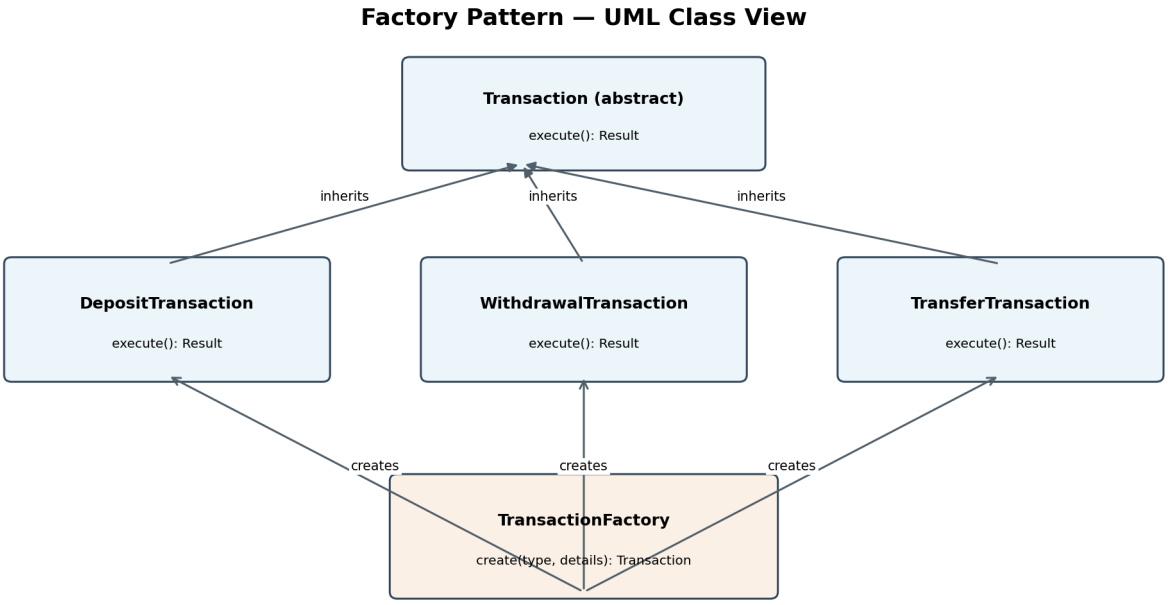
ASSIGNMENT 6

1. Design Pattern Factory Pattern

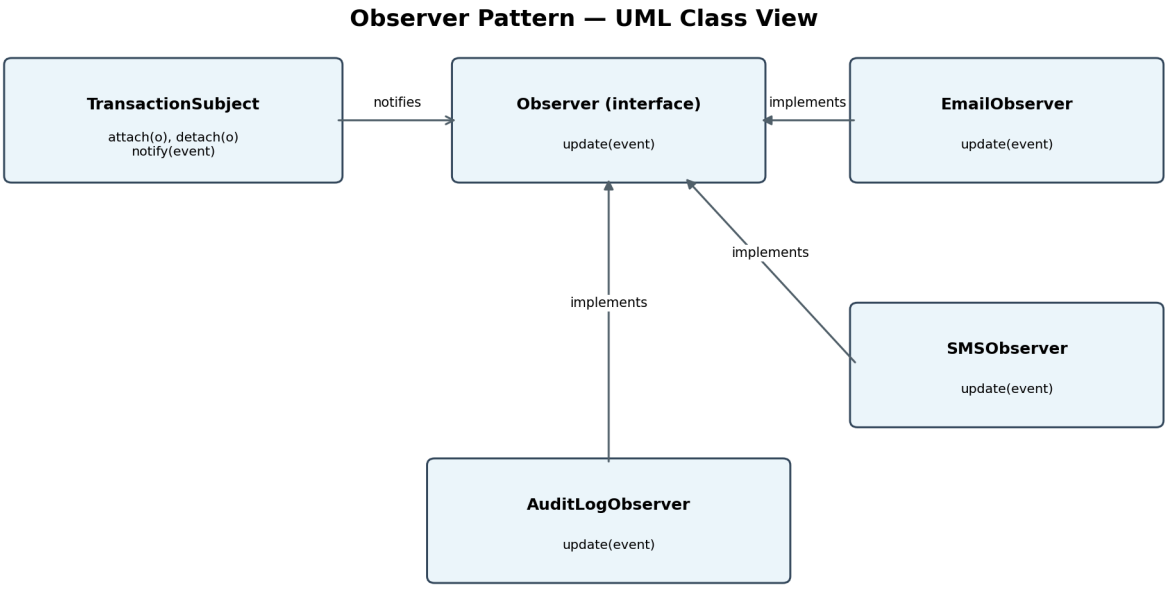
The Factory Pattern centralizes transaction-object creation. BankingApplication asks TransactionFactory for a transaction type such as DEPOSIT, WITHDRAWAL, or TRANSFER. The factory returns the corresponding Transaction implementation, allowing new transaction types to be added without placing construction logic throughout the application.



Participants: Transaction is the common abstraction; DepositTransaction, WithdrawalTransaction, and TransferTransaction are concrete products; TransactionFactory creates the appropriate product based on the requested type.

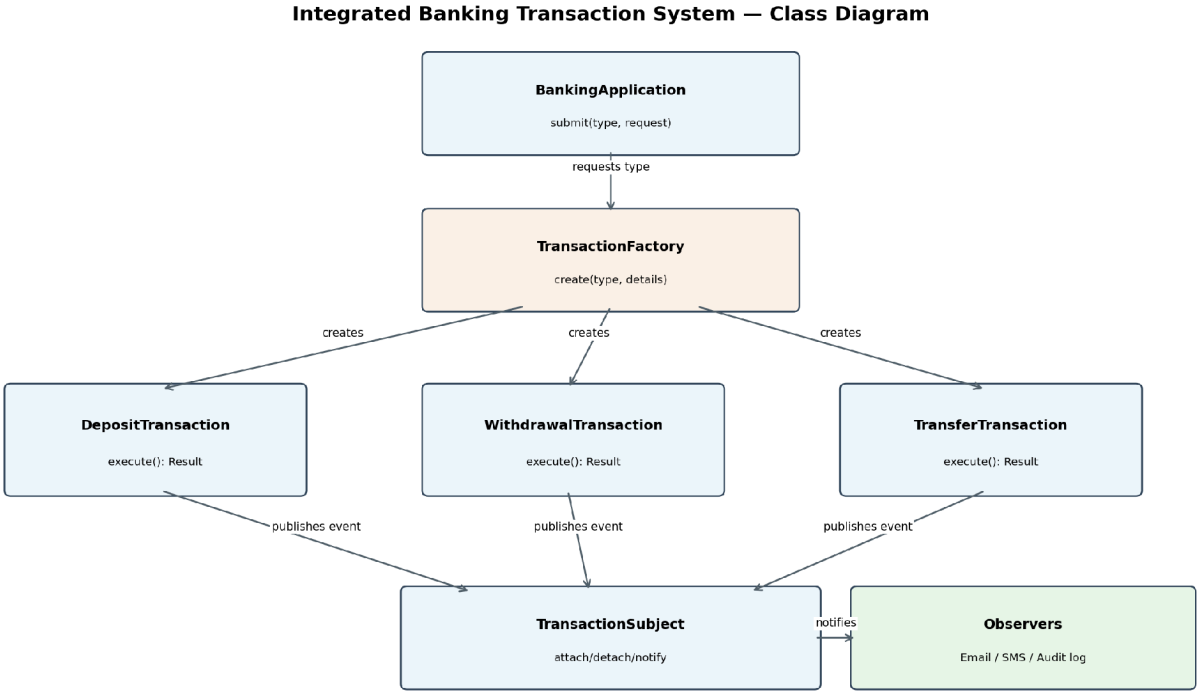
2. Design Pattern Observer Pattern

The Observer Pattern decouples transaction processing from follow-up actions. After a transaction reaches a final state, TransactionSubject publishes an event to subscribed observers. EmailObserver, SMSObserver, and AuditLogObserver independently react to the event. Notification failures should be retried and must not reverse a completed financial transaction.



Participants: TransactionSubject maintains subscribers and sends events; Observer defines update(event); EmailObserver, SMSObserver, and AuditLogObserver are concrete observers.

3. Integrated Class Diagram



BankingApplication uses TransactionFactory to create the selected transaction. Each transaction executes its business operation and publishes a completion event through TransactionSubject. The subject notifies subscribed observers without requiring the transaction classes to know the details of email, SMS, or audit implementations.

4. Pseudocode

```
ENUM TransactionType = { DEPOSIT, WITHDRAWAL, TRANSFER }

INTERFACE Transaction:
    execute(request) -> TransactionResult

CLASS DepositTransaction IMPLEMENTS Transaction:
    execute(request):
        validate amount > 0
        account = AccountRepository.getForUpdate(request.accountId)
        account.balance = account.balance + request.amount
        AccountRepository.save(account)
        return SUCCESS(transactionId, account.balance)

CLASS WithdrawalTransaction IMPLEMENTS Transaction:
    execute(request):
        validate amount > 0
        account = AccountRepository.getForUpdate(request.accountId)
        IF account.balance < request.amount:
            return FAILURE("INSUFFICIENT_FUNDS")
        account.balance = account.balance - request.amount
        AccountRepository.save(account)
        return SUCCESS(transactionId, account.balance)

CLASS TransferTransaction IMPLEMENTS Transaction:
    execute(request):
        validate amount > 0 AND sourceAccountId != destinationAccountId
        lock source and destination accounts in a consistent order
        IF source.balance < request.amount:
            return FAILURE("INSUFFICIENT_FUNDS")
        source.balance = source.balance - request.amount
        destination.balance = destination.balance + request.amount
        save both accounts atomically
        return SUCCESS(transactionId, source.balance)

CLASS TransactionFactory:
    create(type):
        IF type == DEPOSIT: return new DepositTransaction()
        IF type == WITHDRAWAL: return new WithdrawalTransaction()
        IF type == TRANSFER: return new TransferTransaction()
        THROW UnsupportedTransactionType

INTERFACE Observer:
    update(event)

CLASS TransactionSubject:
    observers = set()
    attach(observer): observers.add(observer)
    detach(observer): observers.remove(observer)
    notify(event):
        FOR EACH observer IN observers:
            enqueue observer.update(event)   // asynchronous, retryable

CLASS BankingApplication:
    subject = TransactionSubject()
    factory = TransactionFactory()

    submit(type, request):
        transaction = factory.create(type)
        BEGIN DATABASE TRANSACTION
        result = transaction.execute(request)
        IF result is SUCCESS:
            save immutable transaction record
            COMMIT
            subject.notify(TransactionCompleted(result))
        ELSE:
            ROLLBACK
        return result
```

```
// Example subscriptions
subject.attach(EmailObserver)
subject.attach(SMSObserver)
subject.attach(AuditLogObserver)
result = BankingApplication.submit(TRANSFER, transferRequest)
```